Workshop · Module 06

Resilient *pipelines.*

06 / 08

Agenda

ARC 1 — THE ERROR MODEL

Know what you're catching

01 **The exception hierarchy**
TypeError / ValueError for input; NEXUSError for task failures; structured metadata on both.

02 **trace_id and the exception class**
Every NEXUSError carries a trace_id. The class itself tells you what to do next.

03 **Three buckets, three actions**
fix the input → transient retry → escalate any other NEXUSError.

ARC 2 — THE PATTERNS

Keep the pipeline alive

01 **Exponential backoff with jitter**
Double the wait. Add 20 percent noise. Prevents thundering herds when many clients fail at once.

02 **Resilient poll loop — deadlines and error caps**
Wall-clock deadline, consecutive-error counter. Deadline exceeded means save the id and resume.

03 **Exercise — wrap the full pipeline**
Submit, poll, predict, persist. Simulate a transient failure. Prove it recovers.

By the end of this module, you'll be able to…

Three things you'll *leave with*

A

Read the *exception class* — pick the right response

Input errors mean fix your code. Transient errors mean retry with backoff. Any other NEXUSError means escalate to support. Three buckets, three actions — and the class tells you exactly which.

B

Retry transients with *backoff and jitter*

Write a reusable @with_retry decorator. Exponential delay, 20 percent jitter, caps at two minutes. User errors and internal errors surface immediately — no blind retries.

C

Build a *deadline-based* poll loop

Wall-clock deadline, consecutive-error cap, task id preserved on every failure. Wrap it with submit and predict to ship a pipeline that survives the network.

PART 01

Works in a *notebook*.
Now make it *survive*.

The one-sentence definition.

Every failure is a *typed exception*. The class picks the response — *fix*, *retry*, or *escalate*.

FIG. 06 · THE MODULE IN ONE SENTENCE

Three layers. *Three places* things go wrong.

Where the error is raised decides what you do next. Before you write any retry, know which layer is failing.

01

Client-side *validation*

Your code, before the request leaves. `TypeError` and `ValueError` catch bad inputs — wrong dtype, unknown column, null where a value is required. *Fix and re-run.*

02

Server-reported *failures*

The server ran your task and it failed. A typed `NEXUSError` subclass with `trace_id` and a `details` dict attached — structured metadata, not a string to grep.

03

The *decision rule*

The class picks the action: input errors → fix. transient → retry. any other `NEXUSError` → escalate. The rest of the module is tooling on top of this mental model.

THE ERROR MODEL

Three *buckets*. One decision.

Every NEXUSError carries a *trace_id* and a *details* dict; the exception class itself tells you what to do. TypeError and ValueError cover client-side validation before the call reaches the server.

ValidationError, AuthenticationError, AuthorizationError, NotFoundError mean *fix the input* — do not retry. RateLimitError, NetworkError, RequestTimeoutError, ServerError are *transient* — retry with backoff. Anything else inheriting from NEXUSError — *escalate*: save trace_id and contact support.

THE DECISION RULE

Read the class. Pick the action.

Branch with isinstance(exc, RETRYABLE_EXCEPTIONS) against the typed exception classes — the class is the category.

3 → 3 three categories map to three actions, nothing else.

Backoff, then *decorate*.

One helper. One decorator. Every SDK call becomes resilient.

module_06_resilient_pipelines.ipynb · the backoff and retry-decorator cells

```python
RETRYABLE_EXCEPTIONS = (RateLimitError, NetworkError,
                        RequestTimeoutError, ServerError)

def backoff_delay(attempt, base=5.0, cap=120.0):
    delay = min(base * (2 ** (attempt - 1)), cap)
    delay += random.uniform(0, delay * 0.2)  # jitter
    return delay

def with_retry(max_retries=3, base_delay=5.0):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_retries + 2):
                try:
                    return fn(*args, **kwargs)
                except RETRYABLE_EXCEPTIONS as exc:
                    if attempt > max_retries: raise
                    time.sleep(backoff_delay(attempt, base_delay))
        return wrapper
    return decorator
```

WHY THIS MATTERS

Transients retry. Others surface.

The wrapper catches only RETRYABLE_EXCEPTIONS — isinstance(exc, RETRYABLE_EXCEPTIONS) is the same test. Anything else raises immediately: input errors should not loop, and platform bugs need a human. The decorator keeps that rule in one place.

THUNDERING HERDS

The worst retry storm is the *synchronized* one.

Ten clients all hit a rate limit at the same second. All retry at 5s. Then 10s. Then 20s. They arrive *together*, hit the limit again, and retry in lockstep. The server rate-limits them twice — once from the real overload, once from the retries stacking up behind it.

Jitter breaks the lockstep. A random 20% added to each delay spreads the next wave across a window. The server sees a *trickle* instead of a spike, and the same retries that were compounding the problem now ride the decay out.

THE FIX

20% noise turns a spike into a trickle

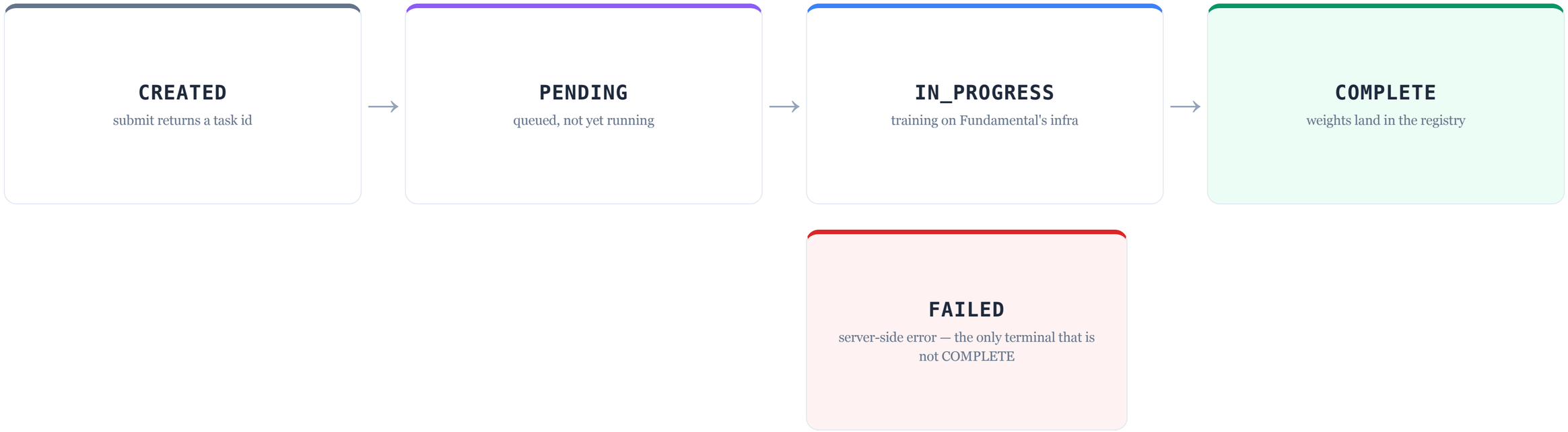
`random.uniform(0, delay * 0.2)` on each attempt. Cheap to add, essential to include. Without it, synchronized failures compound; with it, they dissipate on their own.

20% jitter the difference between recovery and cascading failure.

Now the *poll loop*. Deadlines matter more than retries.

The task *lives on the server*.

Your poll is just a client checking in on this state machine.



Your poll sees **one** *of these states. Client-side errors (NetworkError, RequestTimeoutError) do not change which state the task is in.*

THE INSIGHT

A timeout on poll is *not* a job failure.

A `RequestTimeoutError` during *polling* means the poll call's connection timed out. The task itself kept running. The server-side state machine did not move to FAILED — it stayed wherever it was.

Retry the poll with *the same task id*. Do not re-submit. Re-submission creates a second job, doubles billing, and races the first one. The id you already have is still valid.

THE RULE

The job keeps going. So can you.

RequestTimeoutError is in *RETRYABLE_EXCEPTIONS* — exactly what @with_retry handles. The task id stays valid for 48 hours, so your next poll picks up the state change that was already in flight.

---

same id valid 48h — poll again, never re-submit.

The *resilient* poll.

Wall-clock deadline. Consecutive-error cap. Task id preserved.

module_06_resilient_pipelines.ipynb · the resilient-poll cell

```python
def resilient_poll_fit(nexus, task_id, poll_interval=20,
                       max_poll_errors=5, deadline_seconds=3600):
    start, errs = time.time(), 0
    while True:
        if time.time() - start > deadline_seconds:
            raise RuntimeError(f"deadline exceeded — save {task_id}")
        try:
            result = nexus.poll_fit_result(task_id)
            errs = 0  # reset on clean poll
            if result is not None: return result
            time.sleep(poll_interval)
        except RETRYABLE_EXCEPTIONS as exc:
            errs += 1
            if errs >= max_poll_errors: raise
            time.sleep(backoff_delay(errs, base=10.0))
        except NEXUSError: raise  # non-retryable: fix or escalate
```

WHY THIS MATTERS

Deadlines beat infinite loops.

A transient storm can't trap you — the error counter caps it. A slow job can't trap you — the wall-clock deadline caps it. On deadline exceeded, the task id is in the message, the job keeps running, and a later session resumes the poll.

THE FULL PIPELINE

Submit. Poll. Predict. *Persist.*

run_pipeline is where every pattern in this module clicks into place. Submit the fit task, catch user errors at the input layer, hand the id off to resilient_poll_fit, wrap predict in @with_retry, and write the model_id plus AUC to a registry file.

The invariant: the *task id is saved before the poll*, not after. Poll failures do not kill the job — the server keeps running it. A crashed client can always resume with the id it already wrote to disk.

THE LOAD-BEARING RULE

Save the task id before you poll

Not after. Not on success. Before. The one line that separates a pipeline that survives a restart from one that loses an hour of training.

---

4 steps submit, poll, predict, persist — one function.

EXERCISE

06

Wrap it. Break it. Watch it recover.

FORMAT	Solo, at your laptop
MATERIALS	The notebook, your SLIM_MODEL_ID from Module 05
OUTPUT	A working run_pipeline() + a log showing recovery
DIFFICULTY	Advanced · ●●●●

PARTICIPANT INSTRUCTIONS

Wrap submit + poll + predict into one resilient pipeline. Inject a transient failure. *Prove it recovers.*

01 Build run_pipeline(X_train, y_train, X_eval, y_eval). Use @with_retry on submit and predict, resilient_poll_fit between them.

02 Persist the task id to a registry file *before* the poll call — not after.

03 Monkey-patch poll_fit_result to raise a *transient* NEXUSError on the first two calls. Re-run. Read the logs.

04 In one paragraph: what would change if the failure were an *input error* like ValidationError instead. Paste into the notebook.

Three *takeaways*

If you forget everything else, remember these.

01

The *exception class* picks the action

Three buckets, three responses. Branch with isinstance against the typed exception classes — retry RETRYABLE_EXCEPTIONS, fix the input on the rest, escalate any other NEXUSError.

02

Backoff with *jitter*

Double the wait. Add 20 percent noise. Cap at two minutes. Synchronized clients failing at once is worse than any single client failure — jitter prevents the stampede.

03

Never *discard* a task id

Save the id before you poll, not after. Poll failures do not kill the job. A fresh NEXUSClassifier instance with the saved id resumes the work from anywhere.

NEXUS onboarding | Fundamental

Next — diagnostics.

Module 07 · Verbose logs, diagnose() context, five failure fingerprints · 300-level

RESILIENCE KEEPS THE PIPELINE *RUNNING*. DIAGNOSTICS LET YOU INVESTIGATE WHEN IT DOESN'T.